

Protocol-Aware Observability for AI Chat Streaming: A Systematic Review of Tracing and Metrics for Server-Sent Events and WebSocket

Undergraduate Research Draft

June 2026

Abstract

AI chatbot and retrieval-augmented generation (RAG) systems stream responses token-by-token so users see partial output early. Developers often default to WebSocket, yet for server-to-client text generation, Server-Sent Events (SSE) is frequently sufficient and simpler. Observability, however, is harder for both: REST tracing relies on per-request HTTP headers, but once an SSE or WebSocket connection is established, individual events or messages no longer carry HTTP headers, so request-level instrumentation cannot see token-level behaviour. This paper reports a systematic, PRISMA-guided review of how SSE and WebSocket are compared and observed, focusing on the metrics used for streaming AI workloads. We find that (i) token-level metrics — time to first token (TTFT), inter-token latency (ITL), and output throughput — are well defined in the LLM-serving literature; (ii) transport-level differences between SSE and WebSocket are minor for one-way token streaming; and (iii) no existing work provides a unified, backend-independent, agent-free instrumentation model that captures token-level observability across both protocols. We synthesise these gaps into a five-layer protocol-aware observability framework and a prototype scope for a Java/Spring Boot library. This is a preliminary review: experimental measurement is reported as a planned prototype rather than completed results.

Keywords: Server-Sent Events, WebSocket, observability, distributed tracing, OpenTelemetry, LLM streaming, time to first token, PRISMA.

1 Introduction

1.1 Rationale

Modern chatbot and RAG applications stream responses progressively so that users can read partial output before generation completes. Many developers assume such systems must use WebSocket because it supports bidirectional real-time communication. However, for AI text streaming the dominant pattern is one-way server-to-client delivery of generated tokens, for which SSE can be sufficient and operationally simpler. Both OpenAI and Anthropic expose their streaming APIs over SSE [14, 15].

From an observability perspective, REST APIs are straightforward to trace because every request carries headers such as `traceparent`. SSE and WebSocket, by contrast, are long-lived connections: headers are present only at connection setup, and subsequent events or messages do not automatically carry HTTP headers [19, 11]. Tracing streaming systems therefore requires additional connection- and message-level context propagation. This is a practical gap: existing tools trace REST and database calls well, but tracing the lifecycle of SSE and WebSocket streams — connection open, per-token delivery, disconnects, cross-service propagation — is less direct.

1.2 Objectives and Research Questions

This review studies how SSE and WebSocket are compared and observed for AI streaming, and what gaps remain. It addresses:

1. **RQ1:** What metrics are commonly used to compare SSE and WebSocket?
2. **RQ2:** In AI token streaming, when is SSE sufficient and when is WebSocket more appropriate?
3. **RQ3:** How well do current observability tools support REST tracing compared to SSE and WebSocket tracing?
4. **RQ4:** What gaps exist in message/token-level tracing for long-lived streaming connections?
5. **RQ5:** Can token-level metrics (TTFT, ITL, tokens/s) be captured for both SSE and WebSocket through a single, unified instrumentation model?

1.3 Contributions

(i) A systematic, tiered review of metrics and tracing practice for SSE and WebSocket in AI streaming; (ii) a five-layer protocol-aware observability framework; and (iii) a prototype scope for a backend-independent, agent-free Java/Spring Boot library that instantiates the framework.

2 Background

2.1 REST API Observability

REST observability focuses on request latency, status code, throughput, error rate, and payload size, with distributed tracing achieved by propagating trace context through HTTP headers. Each service extracts incoming context, creates a span, and propagates context downstream. This fits short-lived request–response communication [19, 21].

2.2 Server-Sent Events

SSE pushes events from server to client over a single long-lived HTTP connection using the `text/event-stream` media type and the `EventSource` API [18]. It suits one-way streaming (notifications, dashboards, AI text). After the connection is established, individual events carry no independent HTTP headers, so per-event tracing requires server-side metadata or payload-embedded identifiers [10].

2.3 WebSocket

WebSocket provides a persistent, bidirectional channel established via an HTTP upgrade handshake, after which communication uses WebSocket frames [17]. Because post-handshake frames do not behave like REST requests, tracing WebSocket requires two levels of instrumentation: connection-level and message-level, the latter relying on application metadata such as a trace id embedded in the message envelope [12, 13].

2.4 Distributed Tracing and Trace Context

Distributed tracing tracks how an operation moves across services as a tree of spans [21]. The W3C Trace Context specification standardises `traceparent/tracestate` headers [19], and OpenTelemetry provides a vendor-neutral ecosystem [20]. The standard model works most naturally for HTTP request–response; streaming requires additional design because post-handshake messages may not carry headers.

2.5 AI/LLM Token Streaming

LLM serving is characterised by token-level latency metrics: *time to first token* (TTFT), the time until the first token appears; *inter-token latency* (ITL, also TPOT), the average gap between consecutive tokens; and output *throughput* in tokens per second [7, 8, 6]. Interactive applications benefit most from low TTFT and ITL. In RAG, TTFT additionally includes retrieval time before generation begins.

3 Methods

This review follows the PRISMA 2020 reporting guideline [1] as a lightweight protocol. As an early-stage review, final inclusion counts are provisional pending full-text assessment.

3.1 Eligibility Criteria

Inclusion: sources discussing SSE, WebSocket, HTTP streaming, or long-lived connections; providing latency, throughput, bandwidth, resource, or connection metrics; discussing tracing/telemetry/monitoring; in a streaming, chatbot, AI, or real-time context; or providing implementation detail relevant to Java/Spring Boot/OpenTelemetry.
Exclusion: frontend-only, no metric/observability content, purely promotional, off-topic, or outdated (unless foundational).

3.2 Information Sources

Academic databases (IEEE Xplore, ACM Digital Library, SpringerLink, arXiv, Google Scholar) and authoritative technical sources (OpenTelemetry, Spring, MDN, W3C, IETF, and vendor engineering blogs). All sources were last searched on 23 June 2026.

3.3 Search Strategy

Representative query clusters: (a) ("Server-Sent Events" OR SSE) AND WebSocket AND (performance OR latency OR throughput OR scalability); (b) (WebSocket OR SSE) AND (observability OR tracing OR telemetry OR monitoring); (c) ("LLM streaming" OR "token streaming" OR "AI chatbot") AND (SSE OR WebSocket); (d) (OpenTelemetry OR "distributed tracing") AND (WebSocket OR SSE OR "long-lived connection" OR streaming).

3.4 Selection Process and Data Items

Records were screened by title/abstract then full text against the criteria. For each included source we extracted: id, year, source type, protocol, system context, metrics reported, observability method, trace granularity, strength, and limitation.

3.5 Risk of Bias (Source Tiering)

Because the topic is young and partly documented in industry channels, each source is tiered: **T1** peer-reviewed (conference/journal/thesis); **T2** standard, specification, or authoritative vendor technical documentation; **T3** engineering blog or repository (used as evidence of current practice, not as empirical proof).

3.6 Synthesis Method

Given heterogeneous sources, we use narrative (qualitative) synthesis, grouping evidence by protocol and by metric layer, and answering each research question in turn.

4 Results

4.1 Study Selection (PRISMA Flow)

Figure 1 summarises identification and screening. Counts are provisional: identification reflects the corpus gathered on the search date; final inclusion is to be confirmed after full-text assessment.

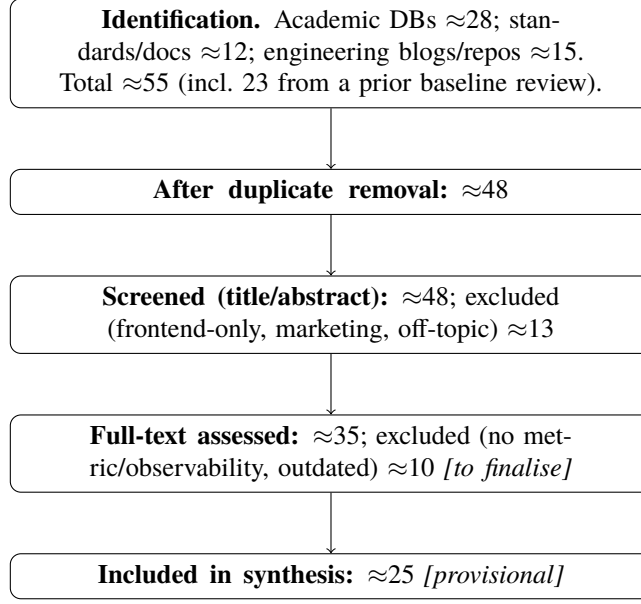


Figure 1: PRISMA 2020 flow (provisional working counts).

4.2 Study Characteristics

Table 1 lists representative included sources with their quality tier.

Table 1: Representative included sources (tiered).

ID	Year	Tier	Focus	Metrics / contribution
WWW’21 WebSocket adoption [2]	2021	T1	WS in the wild (71k connections)	adoption, usage patterns
WS full-duplex eval [3]	2014	T1	WS vs TCP	latency, network traffic
WS data transmission [4]	2019	T1	WS RTC	throughput, latency
WS benchmark infra [5]	2016	T1	server-side benchmark	black-box throughput
LLM serving eval [6]	2025	T1	LLM serving	TTFT, ITL, TPS, pitfalls
NVIDIA benchmarking [7]	2025	T2	LLM serving	TTFT, ITL, TPS defs
Anyscale metrics [8]	2026	T2	LLM serving	ITL formula
OTel GenAI semconv [9]	2026	T2	LLM observability	token-usage attributes
OneUptime SSE [10]	2026	T3	SSE tracing	connection+event span
OneUptime WS [11]	2026	T3	WS tracing	cardinality problem
Alibaba LoongSuite [12]	2026	T3	WS observability	chunk metrics, agent
SSE vs WS for LLM [14]	2026	T3	transport choice	latency negligible

4.3 Synthesis by Research Question

RQ1 — Metrics. Two distinct layers emerge. *Transport-level* (T1 academic): latency, throughput, bandwidth, CPU/memory, TCP connection count, and message frequency [3, 4, 2]. *Token-level* (LLM serving): TTFT, ITL/TPOT, tokens/s, and end-to-end latency [6, 7, 8]. The token-level set, not the REST set, is the relevant comparison basis for AI streaming.

RQ2 — SSE vs WebSocket. Evidence converges: for one-way server-to-client AI text streaming, SSE is sufficient and simpler (automatic reconnection, CDN compatibility, no sticky sessions), and is the choice of major LLM providers. The transport-level latency difference ($\approx 5\text{--}7$ ms) is negligible relative to per-token generation time ($\approx 10\text{--}12$ ms), so the model, not the transport, is the bottleneck [14, 15]. WebSocket is preferable when mid-stream bidirectional input is required (interruption, voice, collaboration) [11, 16].

RQ3 — Tool support. REST tracing is mature. LLM calls now have an emerging standard via the OpenTelemetry GenAI semantic conventions [9], but most attributes are experimental and do not yet cover the streaming lifecycle. SSE and WebSocket tracing still require deliberate, manual instrumentation [10, 11, 12].

RQ4 — Gaps. Two recurring gaps: (i) after the handshake, frames/events carry no HTTP headers, breaking REST-style propagation; (ii) emitting one span per message or per token explodes span cardinality and overwhelms the collector [11]. Both motivate a *two-tier* model: a connection-lifetime span plus a lightweight per-event/token record.

RQ5 — Unified model. The SSE pattern [10] and the WebSocket pattern [11, 12] independently converge on the same two-tier shape, yet no source offers a single, backend-independent, agent-free model that captures token-level metrics across both protocols. This is the gap this work targets.

5 Gap Analysis

REST observability is mature because each request carries headers. SSE and WebSocket lack header-based per-message propagation and risk cardinality blow-up under naive per-message spans. The synthesis indicates a unified two-tier model — one stream span anchoring per-event/token records — applied identically to SSE and WebSocket, and exportable without a mandatory backend, would close the gap left by both agent-based tooling and protocol-specific tutorials.

6 Proposed Protocol-Aware Observability Framework

We propose a five-layer framework (Table 2) and a unified two-tier instrumentation model: each stream (SSE or WebSocket) is one connection-lifetime span, and each token/event is a lightweight *span event* on that span, keeping span cardinality proportional to *streams* rather than *tokens*. Figure 2 shows the end-to-end architecture.

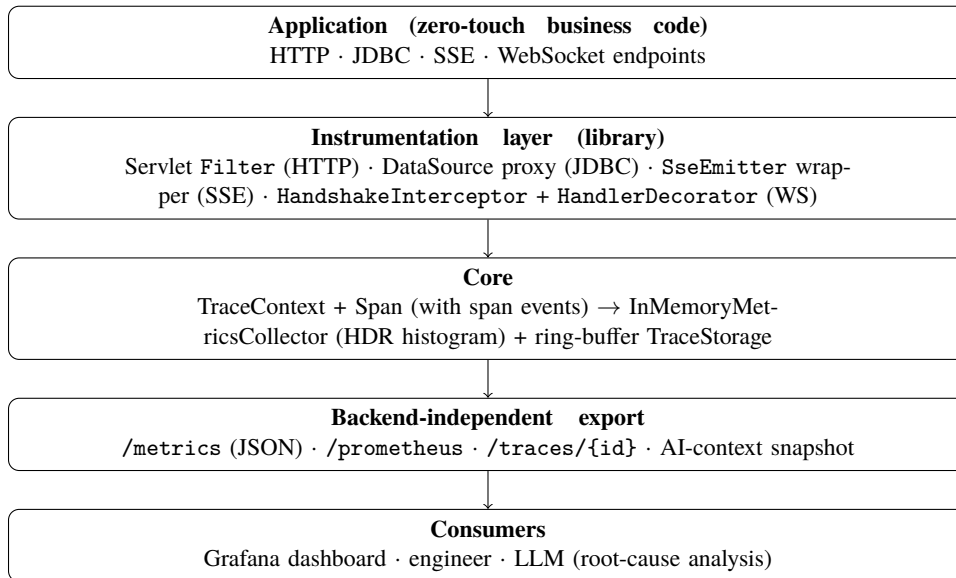


Figure 2: Architecture of the proposed protocol-aware observability library: zero-touch instrumentation feeds an in-memory core that exports without a mandatory backend.

Table 2: Five-layer protocol-aware observability framework.

Layer	Signals
L1 REST/API	request latency p50/p95/p99, throughput, error rate, payload size, status
L2 DB/downstream	JDBC latency, outbound latency, downstream error rate, cross-service span
L3 Streaming connection	active connections, connection duration, disconnect/timeout rate, completion rate
L4 Message/event	messages/events sent/received, chunk index, message size, events per stream
L5 AI streaming	TTFT, total stream duration, token throughput, user abort, provider latency

6.1 Metric Catalogue

We separate *transport-level* metrics (used to compare SSE and WebSocket) from *token/AI-level* metrics (used to characterise streaming quality, applied identically to both protocols). Table 3 is the full catalogue; the core subset measured by the prototype is marked $\star$.

Table 3: Metric catalogue. $\star$ = core subset for the prototype comparison.

Metric	Formula	Unit	Source (tier)
<i>Transport-level</i>			
$\star$ Connection setup	$t_{\text{conn}} - t_{\text{req}}$	ms	RFC 6455; IEEE WS (T1)
$\star$ Active connections	$\# \text{opened} - \# \text{closed}$ (gauge)	count	Alibaba WS (T3)
Stream duration	$t_{\text{close}} - t_{\text{open}}$	ms	OneUptime (T3)
Throughput (bytes)	B_{total}/W	bytes/s	WS thesis (T1)
$\star$ Latency pXX	$\text{percentile}_{XX}(\{d_r\})$	ms	HDR / Micrometer / Prometheus (T2)
Error/disconnect rate	$n_{\text{fail}}/n_{\text{total}} \times 100$	%	streaming refs (T2/T3)
<i>Token / AI-level</i>			
$\star$ TTFT	$t_1 - t_{\text{req}}$	ms	NVIDIA, Anyscale (T2); arXiv (T1)
$\star$ Inter-token latency (ITL)	$\frac{t_N - t_1}{N - 1}$	ms/token	Anyscale, NVIDIA (T2)
$\star$ Output throughput	$\frac{N}{(t_N - t_{\text{req}})/1000}$	tokens/s	NVIDIA, BentoML (T2)
End-to-end latency	$t_N - t_{\text{req}}$	ms	arXiv (T1)
Inter-token jitter	$\text{stddev}(\{t_{i+1} - t_i\})$	ms	derived (smoothness)
Tokens per stream	N	count	OTel GenAI semconv (T2)

Formal definitions. Let t_{req} be the request start time (client side), t_{conn} the moment the connection is established, $t_1, \dots, t_N$ the client receive times of the N streamed tokens, and d_r the per-run latency observed over $r = 1, \dots, R$ runs. The primary token-level metrics are:

$$\text{TTFT} = t_1 - t_{\text{req}}, \quad [\text{ms}] \quad (1)$$

$$\text{ITL} = \frac{t_N - t_1}{N - 1}, \quad [\text{ms/token}] \quad (2)$$

$$\text{Throughput} = \frac{N}{(t_N - t_{\text{req}})/1000}, \quad [\text{tokens/s}] \quad (3)$$

$$\text{Jitter} = \sqrt{\frac{1}{N - 1} \sum_{i=1}^{N-1} ((t_{i+1} - t_i) - \overline{\Delta t})^2}, \quad [\text{ms}] \quad (4)$$

where $\overline{\Delta t}$ is the mean inter-arrival gap. Equation (4) quantifies streaming smoothness: a high jitter at low ITL indicates stuttering output even when average speed is acceptable [6]. Tail latency is reported as percentiles p50/p95/p99 over $\{d_r\}$, computed with an HDR histogram rather than sort-on-query to bound memory and avoid $O(n \log n)$ per query [22, 23].

Measurement notes. Four points must be fixed before any measurement is comparable. **(1) Vantage point.** TTFT and ITL are measured *client-side*, matching LLM-serving benchmark practice [7, 8], because they reflect user-perceived responsiveness. **(2) Controlled mock.** A deterministic generator emits a fixed N with a fixed inter-token delay, so any measured difference between SSE and WebSocket is transport overhead rather than model variance. **(3) ITL definition.** Tools differ on whether the first token is included; we exclude it (Eq. (2) divides by $N - 1$), since TTFT already captures the first-token cost [6, 8]. **(4) SSE latency skew.** An SSE stream is one long-lived HTTP response, so measuring it as a REST “request latency” yields a duration equal to the stream lifetime, distorting p95/p99; latency must therefore be measured per token/event, not per request — a protocol-aware requirement rather than an implementation detail.

6.2 Prototype Scope

A minimal Java/Spring Boot prototype instruments REST (Filter/Interceptor), JDBC (DataSource proxy), SSE (an `SseEmitter` wrapper), and WebSocket (HandshakeInterceptor + WebSocketHandlerDecorator), recording each token as a span event under a single stream span. A deterministic mock token generator (fixed token count and inter-token delay) isolates transport overhead from model latency, enabling a fair SSE-vs-WebSocket comparison on identical input. Metrics are exposed as JSON; no external collector or real LLM provider is required. SSE notably lacks a global instrumentation hook (unlike the Servlet filter or WebSocket decorator), so it must wrap the emitter — itself a finding for RQ3/RQ4.

7 Discussion

Protocol choice should follow workload, not trend: SSE is adequate and simpler for one-way token streaming, while WebSocket is justified by bidirectional interaction. The more consequential issue is observability: streaming systems require protocol-aware, message/token-level instrumentation that header-based REST tracing cannot provide. A backend-independent, agent-free library is attractive for legacy or self-hosted RAG deployments where bytecode agents and full OpenTelemetry backends are undesirable, complementing agent-based solutions such as LoongSuite [12] rather than competing on depth.

7.1 Comparison with Closest Work

Table 4 contrasts this work with the nearest existing approaches across the dimensions a practitioner cares about. No existing approach combines an agent-free, backend-independent design with a single token-level model spanning both SSE and WebSocket.

Table 4: Comparison with the closest existing approaches.

Dimension	OTel + GenAI [20, 9]	LoongSuite [12]	OneUptime [10, 11]	This work
Agent-free	No	No	Yes (manual SDK)	Yes
Backend-independent	No	No	No	Yes
SSE support	Partial	No	Yes	Yes
WebSocket support	Partial	Yes	Yes	Yes
Token-level metrics	Partial	Partial	Partial	Yes
Unified SSE+WS model	No	No	No	Yes
LLM-readable export	No	No	No	Yes

7.2 Scalability Considerations

The two-tier model bounds span cardinality by the number of *concurrent streams*, not by message or token volume: per-stream memory is one span plus lightweight span events, and per-endpoint metrics use fixed-size HDR histograms, so metric memory is independent of request count. Trace storage is a fixed-capacity

ring buffer that evicts the oldest entries. The library is therefore expected to scale comfortably to moderate concurrency; at very high concurrency (e.g. 10^4 – 10^5 simultaneous streams), the per-session context map and active-connection bookkeeping dominate memory and trace sampling becomes necessary, while cross-process aggregation would rely on the optional Prometheus export rather than the in-memory store. These are analytical expectations; quantitative load testing is left to future work.

8 Limitations (Threats to Validity)

The review is preliminary: PRISMA counts are provisional and no benchmark has yet been run. Much of the SSE-vs-WebSocket-for-LLM evidence is T3 (engineering blogs) rather than peer-reviewed — a finding in itself about the immaturity of academic coverage — and is treated as current practice, not proof. TTFT/ITL definitions vary across tools, so any future measurement must state the exact formula used. The proposed framework still requires validation through implementation.

9 Conclusion and Future Work

This preliminary, PRISMA-guided review finds that token-level streaming metrics are well defined, that SSE and WebSocket differ little for one-way token streaming, and that no unified, backend-independent, agent-free model captures token-level observability across both protocols. We propose a five-layer framework and a two-tier instrumentation model, with a Java/Spring Boot prototype scope.

Near-term future work completes the search and screening (finalising the PRISMA counts), implements the SSE adapter and the deterministic mock workload, and reports measured TTFT, ITL, throughput, jitter, and connection metrics comparing SSE and WebSocket under controlled conditions, including a load/scalability test at increasing concurrency. Longer-term extensions include: additional streaming transports (gRPC streaming, HTTP/3 over QUIC, Kafka-based event streaming); tracing for emerging AI-agent protocols such as the Model Context Protocol (MCP) and multi-agent pipelines; an OTLP bridge for interoperability with existing backends; adaptive sampling; and a production deployment study.

References

- [1] M. J. Page et al., “The PRISMA 2020 statement,” *BMJ*, 2021. <https://www.prisma-statement.org/>
- [2] P. Murley et al., “WebSocket Adoption and the Landscape of the Real-Time Web,” *Proc. Web Conf. (WWW)*, 2021. https://faculty.cc.gatech.edu/~mbailey/publications/www21_websocket.pdf
- [3] “Performance evaluation of WebSocket protocol for full-duplex web streams,” *IEEE MIPRO*, 2014. <https://ieeexplore.ieee.org/document/6859715>
- [4] “Performance Analysis of Data Transmission on WebSocket for Real-time Communication,” *IEEE*, 2019. <https://ieeexplore.ieee.org/document/8898135/>
- [5] “Introduction to a WebSocket benchmarking infrastructure,” *IEEE*, 2016. <https://ieeexplore.ieee.org/document/7513661/>
- [6] “On Evaluating Performance of LLM Inference Serving Systems,” arXiv:2507.09019, 2025. <https://arxiv.org/pdf/2507.09019>
- [7] NVIDIA, “LLM Inference Benchmarking: Fundamental Concepts,” 2025. <https://developer.nvidia.com/blog/llm-benchmarking-fundamental-concepts/>

- [8] Anyscale, “Understand LLM latency and throughput metrics,” 2026. <https://docs.anyscale.com/llm/serving/benchmarking/metrics>
- [9] OpenTelemetry, “Semantic conventions for generative AI spans,” 2026. <https://opentelemetry.io/docs/specs/semconv/gen-ai/gen-ai-spans/>
- [10] OneUptime, “Monitor SSE Stream Lifecycle and Delivery Latency with OpenTelemetry,” 2026. <https://oneuptime.com/blog/post/2026-02-06-sse-stream-lifecycle-opentelemetry/view>
- [11] OneUptime, “Trace WebSocket Connections and Real-Time Events with OpenTelemetry,” 2026. <https://oneuptime.com/blog/post/2026-02-06-trace-websocket-connections-realtime-events-opentelemetry/view>
- [12] Alibaba Cloud, “Master WebSocket End-to-end Observability in One Article,” 2026. https://www.alibabacloud.com/blog/master-websocket-end-to-end-observability-in-one-article_602915
- [13] Splunk, “Instrumenting Java WebSocket Messaging,” 2023. <https://community.splunk.com/t5/Community-Blog/Instrumenting-Java-Websocket-Messaging/ba-p/629425>
- [14] “SSE vs WebSockets for Streaming LLM Responses,” 2026. <https://www.buildmvpfast.com/blog/streaming-llm-responses-sse-vs-websockets-2026>
- [15] Procedure, “Why Server-Sent Events Still Win for LLM Streaming,” 2026. [https://procedure.tech/blogs/the-streaming-backbone-of-llms-why-server-sent-events-\(sse\)-still-wins-in-2025](https://procedure.tech/blogs/the-streaming-backbone-of-llms-why-server-sent-events-(sse)-still-wins-in-2025)
- [16] GetStream, “WebSocket vs Server-Sent Events,” 2026. <https://getstream.io/blog/websocket-sse/>
- [17] I. Fette and A. Melnikov, “The WebSocket Protocol,” RFC 6455, IETF, 2011. <https://datatracker.ietf.org/doc/html/rfc6455>
- [18] WHATWG, “Server-sent events,” HTML Living Standard. <https://html.spec.whatwg.org/multipage/server-sent-events.html>
- [19] W3C, “Trace Context,” W3C Recommendation, 2021. <https://www.w3.org/TR/trace-context/>
- [20] OpenTelemetry Documentation. <https://opentelemetry.io/docs/>
- [21] B. H. Sigelman et al., “Dapper, a Large-Scale Distributed Systems Tracing Infrastructure,” Google, 2010. <https://research.google.com/archive/papers/dapper-2010-1.pdf>
- [22] Micrometer, “Histograms and percentiles.” <https://docs.micrometer.io/micrometer/reference/concepts/histogram-quantiles.html>
- [23] Prometheus, “Histograms and summaries.” <https://prometheus.io/docs/practices/histograms/>